

QUESTION 1: Doctor Shift Scheduling using Backtracking Search

Methodology (Methods Used)

The Doctor Shift Scheduling problem is solved using the **Backtracking Search Algorithm**, which is a depth-first search technique used to solve **Constraint Satisfaction Problems (CSPs)**. In a CSP, variables must be assigned values while satisfying a set of predefined constraints.

Method Used: Backtracking Search

Backtracking works by assigning one variable at a time and checking whether the current assignment satisfies all constraints. If a constraint is violated, the algorithm backtracks to the previous assignment and tries another possible value.

Working Procedure

1. Define the variables (Doctors D1, D2, D3).
2. Define the domain (Morning, Afternoon, Night).
3. Assign a shift to D1.
4. Check whether the assignment satisfies the constraints.
5. Assign a shift to D2.
6. Again, verify all constraints.
7. Assign a shift to D3.
8. If all constraints are satisfied, stop.
9. If any constraint is violated, backtrack to the previous assignment and choose another shift.
10. Continue until a valid schedule is obtained.

Constraint Satisfaction

During every assignment, the following constraints are checked:

- D1 cannot work Night.
- D2 must work before D3.
- Only one doctor is allowed in one shift.
- D3 cannot work Morning.
- Every doctor must receive exactly one shift.

The search tree is explored depth-first until a valid solution is found.

Advantages of Backtracking

- Guarantees a valid solution if one exists.
- Efficiently prunes invalid assignments.
- Reduces unnecessary computations.
- Suitable for scheduling and timetabling problems.

Final Result

After applying the Backtracking Search Algorithm, the following valid schedule is obtained:

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Constraint Verification

Constraint	Status
D1 is not assigned Night	✓ Satisfied
D2 works before D3	✓ Satisfied
One doctor per shift	✓ Satisfied
D3 is not assigned Morning	✓ Satisfied
Every doctor assigned exactly one shift	✓ Satisfied

Result Analysis

The Backtracking Search algorithm successfully explored the search space while rejecting invalid assignments. The final schedule satisfies all constraints and demonstrates the effectiveness of Backtracking for solving Constraint Satisfaction Problems.

QUESTION 2: Robot Navigation using Breadth-First Search (BFS)

Methodology (Methods Used)

The robot navigation problem is solved using the **Breadth-First Search (BFS)** algorithm. BFS is an uninformed search algorithm that explores the search space level by level and guarantees the shortest path when all movements have equal cost.

Although the problem mentions the **Manhattan Distance heuristic**, it is used only to estimate the distance from a node to the goal. The actual path selection is performed using BFS.

Method Used: Breadth-First Search (BFS)

BFS uses a **First-In, First-Out (FIFO) Queue** to explore the grid systematically.

Working Procedure

1. Place the Start node into the queue.
2. Mark the Start node as visited.
3. Remove the first node from the queue.
4. Check whether it is the Goal node.
5. If not, expand all valid neighbouring cells.
6. Ignore obstacles and already visited nodes.
7. Insert newly discovered nodes into the queue.
8. Repeat the process until the Goal node is reached.

Role of Manhattan Distance

The Manhattan Distance is calculated as:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

It estimates how many horizontal and vertical moves are required to reach the goal.

Advantages of BFS

- Finds the shortest path when all edge costs are equal.
- Complete algorithm.
- Easy to implement.
- Suitable for grid-based navigation.

Final Result

After applying Breadth-First Search, the robot reaches the destination through the shortest path.

Optimal Path

S

↓

(0,1)

↓

(0,2)

↓

(1,2)

↓

(2,2)

↓

(2,3)

↓

(2,4)

↓

(3,4)

↓

G

Total Number of Moves

8

Cost per Move

1

Total Path Cost

8

Result Analysis

Breadth-First Search successfully explored the grid level by level and avoided all obstacles. The robot reached the destination using the shortest possible path with a total movement cost of **8**.

QUESTION 3: Autonomous Rescue Robot

Methodology (Methods Used)

The rescue robot operates in a **partially observable and dynamic environment**, making it unsuitable for simple search algorithms. Therefore, a combination of **Uniform Cost Search (UCS)** and an **Online Search Agent** is used.

Method 1: Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the **lowest cumulative path cost** rather than the shortest distance.

Working Procedure

1. Start from the initial position.
2. Calculate the cumulative cost of each possible movement.
3. Select the node with the minimum total cost.
4. Expand that node.
5. Continue until the goal is reached.

Because risky zones have additional costs, UCS naturally prefers safer routes.

Method 2: Online Search Agent

Unlike offline search, an Online Search Agent does not know the complete environment in advance.

Working Procedure

1. Sense adjacent cells.
2. Update the internal map.
3. Detect new obstacles.
4. Detect risky zones.
5. Recalculate the optimal path whenever new information is discovered.
6. Continue until the survivor is reached.

AI Concepts Used

Intelligent Agent

The robot behaves as an intelligent agent because it:

- Perceives the environment through sensors.
- Processes environmental information.

- Chooses rational actions.
- Performs movements using actuators.

Rationality

The robot always chooses the action that minimizes the total rescue cost while maximizing safety.

Environment Type

Property	Type
Observability	Partially Observable
Determinism	Stochastic
Environment	Dynamic
Agent	Single Agent
State Space	Discrete

Advantages

- Adapts to unknown environments.
- Avoids dangerous areas.
- Guarantees minimum path cost.
- Suitable for real disaster rescue operations.

Final Result

The rescue robot successfully reaches the survivor by combining **Uniform Cost Search** with an **Online Search Agent**.

Achievements

- Successfully avoided blocked paths.
- Detected and avoided high-risk zones whenever possible.
- Updated its knowledge dynamically as new information became available.
- Selected the safest and least-cost path.
- Successfully reached the survivor.

Performance Evaluation

Performance Measure	Result
Safe Navigation	✓ Achieved
Minimum Path Cost	✓ Achieved
Dynamic Environment Handling	✓ Achieved
Obstacle Avoidance	✓ Achieved
Survivor Reached	✓ Achieved

Result Analysis

The combination of **Uniform Cost Search** and an **Online Search Agent** proved highly effective for disaster-response scenarios. The robot continuously adapted to changes in the environment, minimized travel cost, and safely reached the survivor despite limited environmental knowledge.